

CHAPTER 16

Simulation

16.1. Space story

Quote from a simulation engineer, “The only thing you can be certain about with an analog computer is that the answer it produces will be different tomorrow.”

16.2. Introduction

Simulations are a critical part of spacecraft development. In many industries, experimentation is the key to developing new technologies. For example, circuit designers build prototype circuits on breadboards, which are easy to change and do not require soldering. Changing component values is easy. A wide range of hardware, such as voltmeters, oscilloscopes, etc. is available for testing. Once the designer is happy with the circuit, a printed-circuit board is built and a soldered version is produced. This process continues until production.

In the earliest days of aviation, flight test was the major method for proving new designs. However, even back in the days of the Wright brothers, the idea of a wind tunnel caught on where engineers could study aerodynamics without killing themselves. Testing new aircraft still had to be done by flying the aircraft, which was a very dangerous occupation.

Many industries used scale models to test out ideas. Automobile manufacturers build wood versions of cars before prototyping. CAD software has not replaced scale models.

Spacecraft presented a whole new problem. First, most spacecraft were one-offs, that is, often you only built one. Exceptions were the crewed spacecraft of the 1960s and 1970s and the ISS transfer vehicles. Even when you were able to fly a series of satellites, like DMSP, the amount of data you could get back from the flight was limited. In any case, you always wanted the first one to work as advertised due to the cost of getting it into space. Testing satellites on the ground before a flight is difficult. Some companies use air-bearing tables, with up to three degrees-of-freedom, but they still do not approach actual spaceflight. Building sensor sources, like star fields, planetary horizons, and the Sun, become whole projects in and of themselves.

Before the advent of the digital computer, the first thing one could do is find analytical solutions. For control systems, a huge number of tools were developed for analyzing control systems. Bode, root-locus, Nyquist plots, etc. provide insight into the performance of a linear system. Nonlinear systems could be analyzed using describing functions. Graphical methods were particularly useful because they would provide

a range of information, rather than just a number. For any system, analytical solutions were valued. Mission planning used, and still uses, analytical methods like Kepler propagation and Lambert's law. Patch conics is a way of doing multibody orbit trajectories by hand. Drafting was a critical element that allowed designers to visualize their designs.

Even analytical methods required a lot of computing. Slide rules sped things up but inevitably engineering organizations had to hire rooms full of people, known as computers to do all the number crunching. One of the important math lessons in elementary and high school was learning to use tables of trigonometric functions that engineers, and computers, had to employ.

Analog computers, using electrical circuits as analogs to physical systems, were one approach. Suppose we have a first-order differential equation we wish to simulate

$$\tau \frac{dx}{dt} + x = u \quad (16.1)$$

where u is applied at $t = 0$. This has an analytical solution

$$x = u \left(1 - e^{-\frac{t}{\tau}} \right) \quad (16.2)$$

Fig. 16.1 shows an operational amplifier configured as an analog integrator.

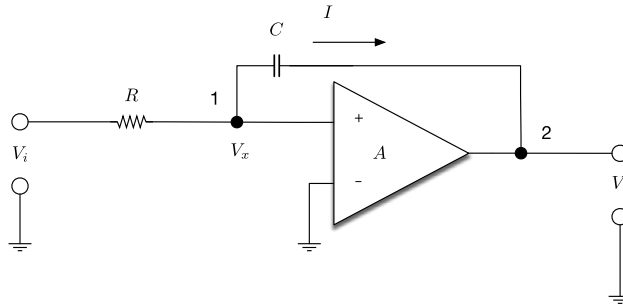


Figure 16.1 Operational amplifier configured as an integrator.

An operational amplifier is a differential amplifier with a very high gain A that amplifies the difference between the plus and minus terminals. The amplifier is so high gain that no current goes into it. The voltage drop across the resistor is

$$V_x - V_i = IR \quad (16.3)$$

The current through the capacitor, C is

$$I = C \frac{d(V_x - V_o)}{dt} \quad (16.4)$$

Since no current is flowing into the op-amp, V_x must be at ground, i.e., 0, where V is the drop across the capacitor. From the voltage drops

$$I = -\frac{V_i}{R} \quad (16.5)$$

or

$$RC \frac{dV_o}{dt} = V_i \quad (16.6)$$

where RC is the time constant, τ . Let $V_o = x$ and $V_i = u$ and we have the integrator part of our differential equation,

$$x_i = \frac{1}{\tau} \int u \, dt \quad (16.7)$$

To create our analog computer integrate Eq. (16.1)

$$\tau x + \int x = \int u \quad (16.8)$$

or

$$x = \frac{1}{\tau} \int u - \frac{1}{\tau} \int x \quad (16.9)$$

The analog computer is shown in Fig. 16.2.

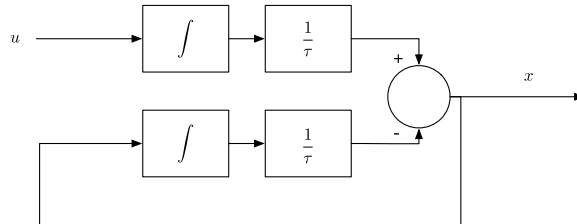


Figure 16.2 Analog computer.

Once you have blocks like integrators, adders, and multipliers you can build a model of any linear system. You can add nonlinear blocks to model nonlinear elements.

The introduction of digital computers made life easier, and harder. The first problem encountered in the 1960s was that the word length was limited, limiting accuracy. The second problem was that floating-point arithmetic was not standard. Not only that, but round-off methods varied and were sometimes done in very strange fashions. Eventually, this was standardized in IEEE-794 [1] by the Institute for Electrical and Electronics Engineers (IEEE), and modern processors when they are made correctly, produce very reliable results.

16.3. Digital simulation

16.3.1 Numerical errors

The analog computer was supplanted by the digital computer in the late 1960s and early 1970s. The fundamental building block of digital simulation is the numerical integrator. Starting with our single integrator

$$\tau \frac{dx}{dt} + x = u \quad (16.10)$$

we can make a numerical simulation as follows.

$$\tau \frac{\Delta x}{\Delta t} + x = u \quad (16.11)$$

$$\Delta x = \frac{\Delta t}{\tau} (u_k - x_k) \quad (16.12)$$

$$x_{k+1} = x_k + \Delta x \quad (16.13)$$

$$x_{k+1} = x_k + \frac{\Delta t}{\tau} (u_k - x_k) \quad (16.14)$$

This simple algorithm is known as Euler's method. This is a discrete version of the continuous differential equation. Δt is the time step and k is the step. Errors arise in two ways. The first is due to the time step. The smaller the time step the more accurate the integration. Therefore there is an error that grows with the size of the time step. Another error is round-off error. Every time an operation is done there will be round-off error since no arithmetic operation is perfectly accurate. The round-off error will grow as more operations are performed. To simulate over a period of time, T requires $n = \frac{T}{\Delta t}$ time steps. The smaller the time step, the more operations and the greater the accumulated round-off error. Smaller time steps reduce truncation error but increase round-off error. With double-precision arithmetic and IEEE standard rounding, round-off error is much less of a problem than it once was. However, one must take care when using an older processor, like the MIL-STD-1750A, which does not use IEEE standard floating-point arithmetic. Appendix A has more on numerical integration methods.

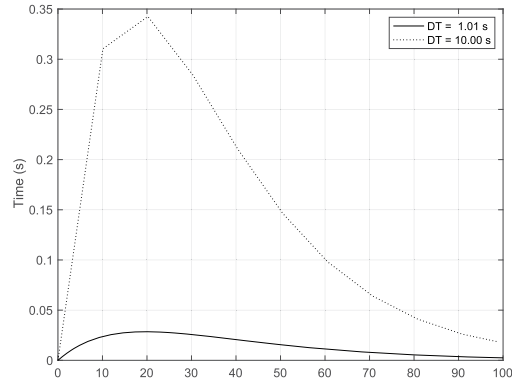
Example 16.1 shows the integration errors when integrating with a one-second time step and a 10-second time step with a time constant, τ of 20 seconds. The simulation is a step response.

Not surprisingly, the 10-second time step has much larger errors. In both cases, the errors are largest when x is changing fast. The simulation is having trouble keeping up with the true state. As time progresses the error drops because the next state is nearly the same as the current state. This shows that one needs to be careful about interpreting simulation results because the numerical errors will be worse when interesting things are happening.

```

1
2 n = 100;
3 t = linspace(0,100,n);
4 tau = 20;
5 u = 3;
6 x = u*(1-exp(-t/tau));
7 dT = t(2);
8 x1 = zeros(1,n);
9 for k = 2:n
10     x1(k) = x1(k-1) + (dT/tau)*(u - x1(k-1));
11 end
12 s1 = sprintf('DT=%5.2f s',dT);
13
14 n = 10;
15 dT = 10;
16 x2 = zeros(1,n+1);
17 for k = 2:n+1
18     x2(k) = x2(k-1) + (dT/tau)*(u - x2(k-1));
19 end
20 s2 = sprintf('DT=%5.2f s',dT);
21
22 NewFig('Simulation')
23 plot(t,abs(x-x1))
24 hold on
25 k = [1:10:100 99];
26 plot(t(k),abs(x(k)-x2))
27 legend(s1,s2)
28 grid
29 ylabel('Error')
30 ylabel('Time(s)')

```



Example 16.1: Euler integration with two different time steps.

16.3.2 Model truncation

We will use a one-axis, two-disk spacecraft to study model truncation error. This is shown in Fig. 16.3. Model truncation may be deliberate, reducing the order of a model to make simulations run faster, or due to modeling errors.

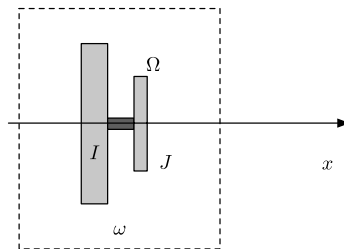


Figure 16.3 One-axis simulation. Two disks are aligned on the same shaft. The dotted line surrounds the entire system.

The angular momentum is

$$H = I\omega + J(\Omega + \omega) \quad (16.15)$$

where I and J are the inertia of the two disks, ω is the angular rate of the entire assembly, and Ω is the angular rate of the second disk with respect to the first. The equations of motion are

$$T = I\dot{\omega} + J(\dot{\omega} + \dot{\Omega}) \quad (16.16)$$

$$T_a = J(\dot{\omega} + \dot{\Omega}) \quad (16.17)$$

where T is the external torque and T_a is the torque between the two disks. T_a is an internal torque. In matrix form this becomes

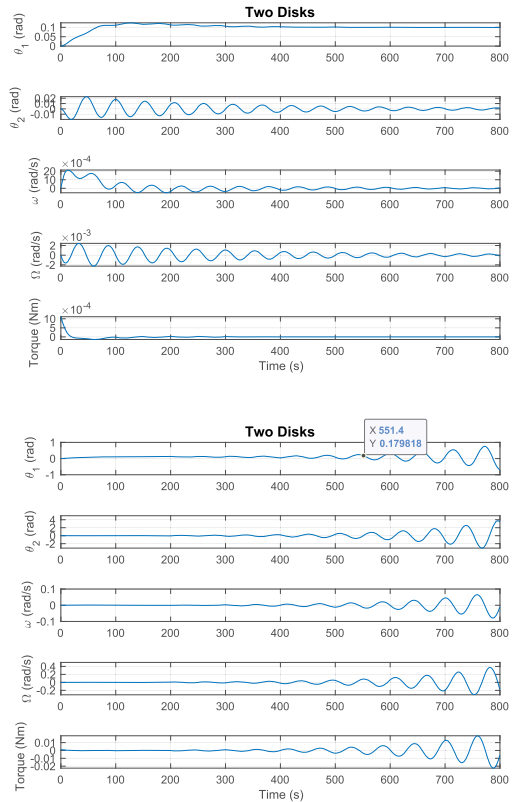
$$\begin{bmatrix} I+J & J \\ J & J \end{bmatrix} \begin{bmatrix} \dot{\omega} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} T \\ T_a \end{bmatrix} \quad (16.18)$$

Example 16.2 shows the two-disk simulation with a lightly damped connection between the disks. A proportional-integral-derivative (PID) controller is used to do an angle change.

```

1 n = 4000;
2 xP = zeros(5,n);
3 dT = 0.2;
4 sensorOnBody2 = false;
5
6 torque = 0;
7 inr1 = 3;
8 inr2 = 4;
9 xPID = [0;0];
10
11 x = [0;0;0;0];
12 xSet = 0.1;
13
14 [a, b, c, d] = PIDMIMO( inr1+inr2, 0.7071, 0.01,
15 1000, 0.1, dT );
16 dRHS = struct('inr1',3,'inr2',1,'k',0.01,
17 'c',0.001);
18 for k = 1:n
19     if ( sensorOnBody2 )
20         err = x(1) + x(2) - xSet;
21     else
22         err = x(1) - xSet;
23     end
24     xPID = a*xPID + b*err;
25     dRHS.t1 = -c*xPID - d*err;
26     xP(:,k) = [x;dRHS.t1];
27     x = RK4(@RHS,x,dT,0,dRHS);
28 end
29
30 t = (0:n-1)*dT;
31 yL = {'\theta_1(rad)', '\theta_2(rad)', '\omega
32      (rad/s)', '\Omega(rad/s)', '\Torque(Nm)'};
33 Plot2D(t,xP,'Time(s)',yL,'TwoDisks');
34
35 %% Right hand side [ang1;ang2;omegal;omega2]
36 function xDot = RHS(x,~,d)
37
38 inr = [d.inr1+d.inr2 d.inr2;d.inr2 d.inr2];
39 t2 = -d.k*x(2) - d.c*x(4);
40 omegaDot = inr\d.t1;t2];
41
42 xDot = [x(3:4);omegaDot];
43
44 end

```



Example 16.2: Two disks with a PID controller.

Two cases are shown. In one, the sensor is on the main body and in the second, the sensor is on body 2.

In the first case, the second disk oscillates with respect to the first. The first disk is perturbed by the oscillation. The PID control design assumes that the spacecraft is rigid. The second disk adds some oscillations to the main-body response but does not change the maneuver much. A bigger issue is that a device mounted to the second disk would oscillate. This motion, known as jitter, could degrade the performance of the device.

In the second case, the control system is unstable. Its design does not take into account the flexible connection between actuation and sensing.

In summary, we observe

1. A small degradation in the spacecraft maneuver, if the actuator and sensor are collocated;
2. Instability if the sensor and actuator are not collocated;
3. An oscillation that may impact the payload;
4. Possible structural fatigue.

Changing the model parameters will change the results.

16.4. Applications of simulation

16.4.1 A sequence of simulations for ACS development

Fig. 16.4 shows one possible simulation sequence for the development of an ACS. Maneuver analysis and disturbance analysis are done first to provide inputs to the simulations. Attitude determination, momentum management, and control simulations are done independently and in parallel using simple simulations. The dynamic models need to reflect what is important to each system. For example, if the spacecraft is flexible and the attitude control system must compensate for the flexibility, then the dynamics model for the attitude control simulation must incorporate a flexible-body model. On some spacecraft slosh is significant and then it also needs to be considered. Within the attitude control block, there might be two or three separation simulations. Momentum management generally is just dealing with inertial momentum so a single integrator usually works.

In the next step, momentum management and attitude control are integrated into a single simulation. The highest-fidelity dynamics model from the attitude control block becomes the baseline dynamics model. If the spacecraft has a propulsion system, it is added at this point. Propulsion-system nonlinearities, like minimum impulse bit, etc. are included at this point. The orbit model is added. A point-mass planet model is usually sufficient. This allows engineers to determine the impact of orbital position on attitude control.

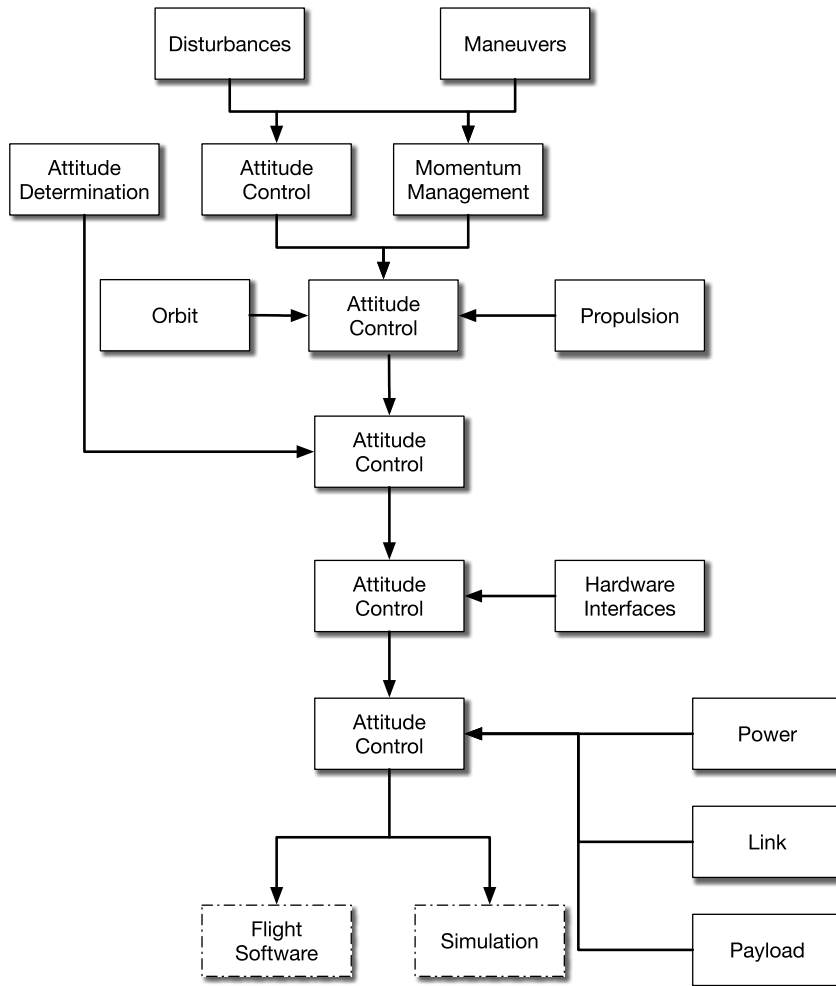


Figure 16.4 Simulation sequence for ACS development. The attitude control block evolves and gains complexity during the design process.

The next step is to add the attitude determination system. Now, the control system is seeing the output of the attitude determination algorithms. Engineers can then easily see the impact of the “real” measurements on the control system.

The next step is to add binary-level interfaces. The interface software converts outgoing torque demands into the required bit patterns for the hardware. Attitude determination receives the simulated words from the sensor. The handling of star cameras is problematic. A full camera model is generally impractical, so a pinhole-camera model is typically used.

The next step is to model the communications, payload, and power systems so that the impact of attitude control on those subsystems. Simulations are used to determine the dynamical effects.

Finally, the flight software is split and loaded onto an equivalent flight processor. The simulation runs on any computer. The hardware models in the simulation should be connected to the same network as on the spacecraft. At this point, the performance of the attitude control system can be assessed. Other subsystem software, for power, payload, and communications, will also be added so that a full flight-software model can be tested.

16.4.2 Analysis support

The simulations in this book are examples of simulations for analysis support. The simulation is used to verify analysis results. This is a good way of catching errors. In many cases, you would write custom code just to verify your analysis results. Since these short simulations usually run quickly, you can do Monte Carlo studies. Monte Carlo studies are running large numbers of simulations with random initial conditions and parameters. When using such simulations they need to be documented and the results recorded to help you study results from more complex or higher-fidelity simulations. One advantage of such simulations is that they can be analytically verified. Some examples are given in the following sections.

16.4.2.1 Sizing of a reaction wheel

Assume the reaction wheel is sized for a particular maneuver. A simulation is built and run to verify the momentum and torque requirements. Complex attitude profiles can be simulated, including profiles that may not be in the baseline operations.

16.4.2.2 Disturbance modeling

Attitude disturbances are computed for a given orbit and attitude profile. A simulation could be run to verify the numbers. If a Fourier series was generated for the profile and orbit it could be compared with the disturbance model itself.

16.4.2.3 Control design

Step and impulse responses could be generated for the controller to verify that the gains get the desired response. Actuator limits and the response to those limits can be tested. The effect of noise filters can be verified by adding noise of varying standard deviations and spectrum to the inputs. Single-axis simulations are a good starting point. These can be followed by rigid-body simulations. If reaction wheels are used, running with and without friction is a good idea.

For thruster systems, you can first look at the ideal response to your thruster control system. You can then add the thruster minimum pulsewidth, pulse resolution, and impulse versus pulsewidth models.

16.4.2.4 End-to-end testing

End-to-end testing means testing the ACS completely as it would be used in orbit. This means

1. Processing commands;
2. Running ACS;
3. Observing the results in telemetry.

Fig. 16.5 shows the flow. Even a fully autonomous system is initiated from the ground, or perhaps a switch on separation. Command processing takes data from the command stream and converts it into engineering units for the ACS. If a sequence of commands has been uploaded, it provides the timing mechanism for the commands. This type of simulation will catch many problems. For example, as part of the design, ACS engineers will specify what parameters can be uploaded as commands. It may be necessary to upload parameters, or other commands in a specific order. End-to-end testing can determine what happens if this is not the case, or catch unanticipated problems with updated control-system parameters.

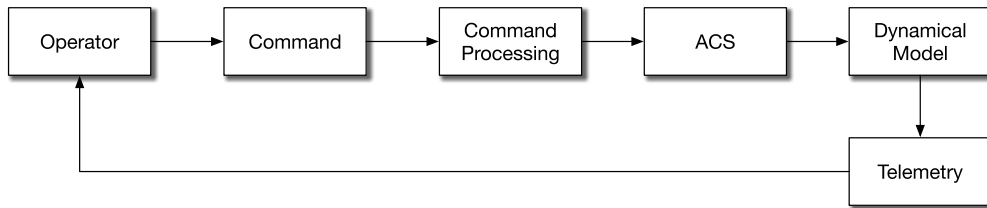


Figure 16.5 End-to-end testing involves the entire operational chain.

An end-to-end simulation can be used as part of operator training, as discussed in Section 16.4.5. Since operators deal with all of the spacecraft subsystems, additional models are usually added.

16.4.3 Performance verification

16.4.3.1 Single case

When debugging a control system it is good to start with the same conditions for each test. This means the same initial states and set the random-number generators to the same initial value. This makes it easy to find problems and compare results for different control-system gains, etc.

16.4.3.2 Grid test

A grid test is what is done after the single case stage is complete. The grid is a small set of initial states and parameters. The initial states should be representative of the conditions expected in flight. The simulation runs for those cases. If a particular case causes a problem, then it can be repeatably run until the problem is solved. Each time a problem is found, the initial conditions should be saved for future testing. The grid test includes variations in parameters that are uncertain. This can include noise levels for sensors, friction levels for reaction wheels, and impulse bits for thrusters. For hardware that has multiple copies on the spacecraft, it is a good idea to vary the parameters among the devices.

16.4.3.3 Numerical gain and phase margins

Typically, a control system is required to meet gain and phase margins. The gain margin is how much the forward gain can change before the system goes unstable. The phase margin is how much the phase can change before the system goes unstable. The phase margin is related to the delay margin in which the quantity of interest is how large a delay the system can tolerate. Some control systems, such as the phase-plane systems used on the Space Shuttle Orbiter [2], are not designed using linear control techniques and are not designed with specific gain and phase margins.

There are several ways to compute margins. One is to use describing functions [3]. The describing-function approach is shown in Example 16.4. A simulation of the phase-plane controller is first run to show that it works very well. The thick green line shows the optimal trajectory. It tracks downward in the phase plane until the first switching line is reached. It then coasts horizontally and finally follows the second switching line to the origin. A very small time step is used to prevent overshoot.

Another method is to vary the phase and gain of signals entering the controller within the simulation. Gain variation is straightforward. Phase variation can be done by adding delays or by adding a phase lag in the control loop. For delays that are a fraction of a control period, the simulation needs to run faster than the control period. Example 16.3 shows delays that are multiples of the control period. A five-second delay makes the system unstable.

This can be applied to a nonlinear control system. Example 16.4 shows a phase-plane controller with and without a delay. The second simulation is run twice as long to show the limit cycle at the origin. The delay causes it to miss the switching lines.

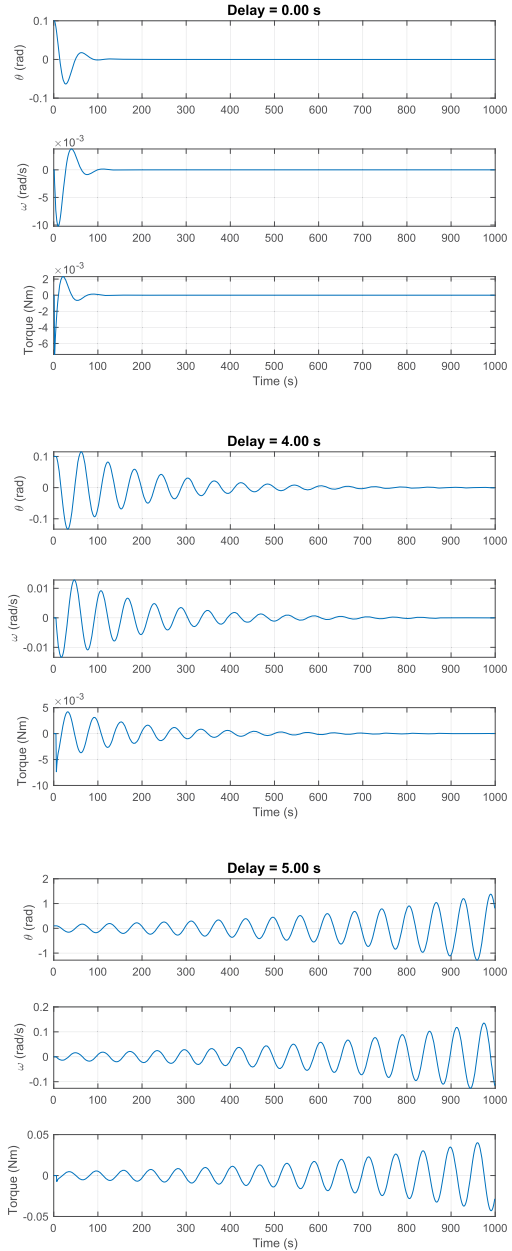
16.4.3.4 Monte Carlo

Monte Carlo simulation is a verification method that uses random inputs. Monte Carlo simulation is named after the well-known casino town in Monaco [4]. Rather than use a

```

1  n          = 1000;
2  xP         = zeros(3,n);
3  dT         = 1;
4  xPID       = [0;0];
5  x          = [0.1;0];
6  dRHS       = struct('inr',3,'torque',0);
7  nDelay     = 6;
8
9  [a, b, c, d] = PIDMIMO(dRHS.inr, 0.7071, 0.1,
10 40, 0.1, dT);
11 tD         = zeros(1,nDelay);
12 for k = 1:n
13     xPID      = a*xPID + b*x(1);
14     xP(:,k)   = [x;dRHS.torque];
15     dRHS.torque = tD(1);
16     torque    = -c*xPID - d*x(1);
17     tD(1:nDelay-1) = tD(2:nDelay);
18     tD(nDelay)    = torque;
19     x         = RK4(@RHS,x,dT,0,dRHS);
20 end
21
22 t = (0:n-1)*dT;
23 yL = {'\theta(rad)', '\omega(rad/s)', 'Torque(Nm)'};
24 s = sprintf('Delay=%g%,4.2f s',(nDelay-1)*dT);
25 Plot2D(t,xP,'Time(s)',yL,s);
26
27 %% Right hand side [ang1;ang2;omegal;omegal2]
28 function xDot = RHS(x,~,d)
29
30 xDot      = [x(2);d.torque/d.inr];
31
32 end

```

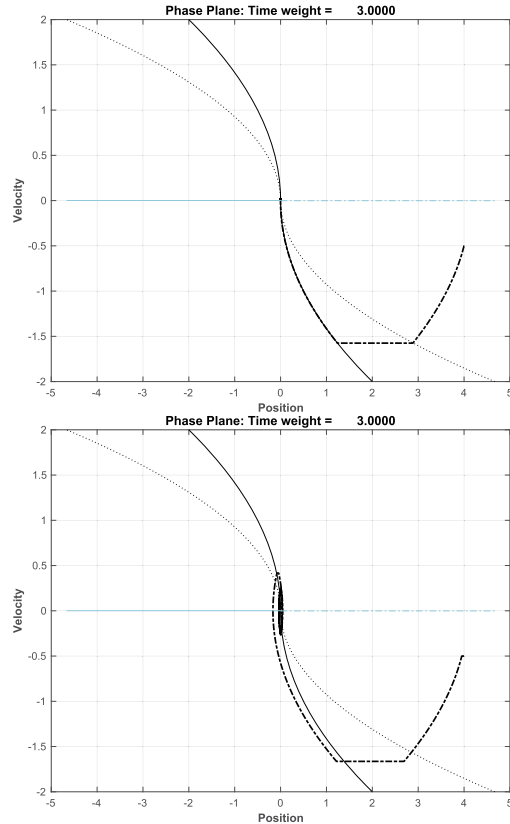


Example 16.3: Control response with different delays. With a 5-second delay, the system is unstable.

```

1 n = 8000;
2 xP = zeros(3,n);
3 dT = 0.001;
4 x = [4;-0.5];
5 dRHS = struct('inr',3,'u',0);
6 nDelay = 100; % 1 for no delay
7 tD = zeros(1,nDelay);
8 kOpt = 3;
9
10 for k = 1:n
11     u = PhasePlane(x(2),x(1),kOpt,0.001,0.001);
12     xP(:,k) = [x;u];
13     dRHS.u = tD(1);
14     tD(1:nDelay-1) = tD(2:nDelay);
15     tD(nDelay) = u;
16     x = RK4(@RHS,x,dT,0,dRHS);
17 end
18
19 PhasePlanePlot( xP(2,:), xP(1,:), kOpt, 0.001, 0.001, [-2,2] )
20
21 %% Right hand side [ang1;ang2;omega;omega2]
22 function xDot = RHS(x,~,d)
23
24 xDot = [x(2);d.u];
25
26 end

```



Example 16.4: Phase-plane controller. The first plot shows the phase-plane trajectory in green without delay. The switching lines are also shown. The second shows it with the delay.

deterministic set of inputs, the initial conditions and model parameters are chosen from a probability distribution. Besides possibly catching errors, it allows you to do sensitivity analyses and determine the correlation of inputs.

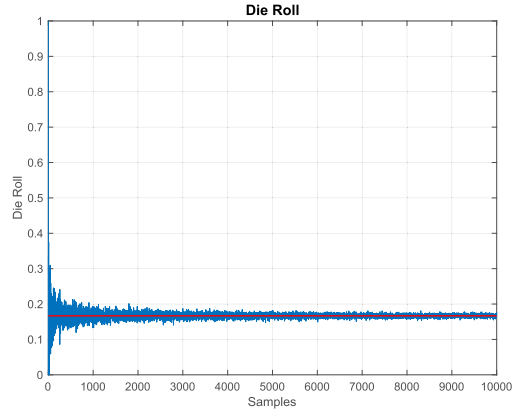
It is interesting to get a feel for the number of cases needed for a Monte Carlo simulation process. Example 16.5 gives the example of a single dice. The probability of any number is 1/6. Converging to that value takes many samples.

Even for this simple case, nearly a thousand tests are needed to converge on the known probability of a given digit resulting from the throw of the dice. The idea of the Monte Carlo simulation is to pick random samples that cover the range of possible initial conditions and parameters without having a computationally impossible number of cases.

```

1 kMax = 10000;
2 a = zeros(1,kMax);
3 for k = 1:kMax
4     r = randi(6,1,k);
5     j = find(r==1);
6     a(k) = length(j)/k;
7 end
8
9 Plot2D(1:kMax,a,'Samples','Die_Roll')
10 line([1 kMax],[1/6 1/6],...
11     'linewidth',2,'color',[1 0 0]);

```



Example 16.5: Die roll. The probability that a dice roll will be a given number is $1/6$. It takes many trials to converge to that value.

16.4.3.5 Commands

Most ACS are commandable. You can upload new gains and send commands to actuators using commands from the ground. These need to be exercised. Commands must be shown to mesh with the other ACS operations. Incorrect sequences of commands, such as turning on a hydrazine thruster before turning on the catalyst-bed heater, should produce the correct response.

16.4.3.6 Edge cases and stress cases

Edge and stress cases are when the sensors and actuators are operating at their operational limits. For example, a spacecraft can be given a maneuver with the starting value of the reaction-wheel inertia near its maximum value. Another case is when a star sensor enters the Sun or Earth stay-out zone. Operations that transfer from one sensor to another should be tested. If your spacecraft has attitude control modes, transitions between modes should be simulated. For a spacecraft that is entering a planetary atmosphere, the maximum allowable entry conditions should be exceeded in testing. This is especially important for landers on other planets or moons where the atmosphere model will not be as well known as the Earth's.

16.4.3.7 Failure cases

Failure cases need to be simulated to understand the dynamics of a failure. For example, a pointing-control system that uses an Earth sensor will not point correctly if the Earth sensor fails. The response to a failure may result in operational problems beyond the control system not pointing correctly. To simulate a failure it is necessary to understand

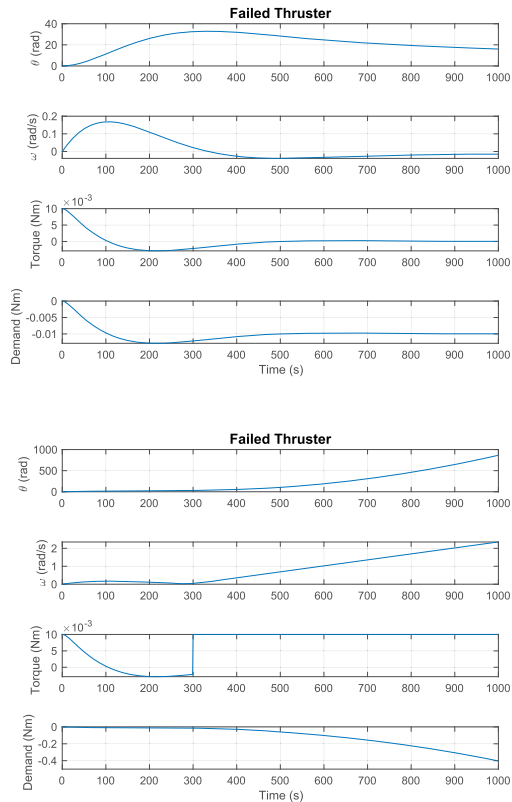
the failure modes of the devices. For example, thruster valves are generally designed to fail closed as failing open would result in a continuous torque on the spacecraft.

Example 16.6 shows an example of a spacecraft with two thrusters. It has a PID control law. When one thruster fails, the torque can only have one sign. In this case, the disturbance is positive so a failed positive failed sign is unobservable. If the failure sign is negative, the attitude diverges because the total torque is just the disturbance. As this shows, it is necessary to test the response for all thruster failures, and also to run simulations with different disturbances and initial conditions.

```

1 % Failed thruster simulation
2
3 n = 1000;
4 xP = zeros(4,n);
5 dT = 1;
6 kFail = 300;
7 failSign = 1;
8
9 torque = 0;
10 inr = 3;
11 xPID = [0;0];
12
13 x = [0;0];
14 dist = 0.01;
15
16 [a, b, c, d] = PIDMIMO( inr, 0.7071, 0.01, 1000,
17     0.1, dT );
18 for k = 1:n
19
20     % Controller
21     xPID = a*xPID + b*x(1);
22     torqueD = -c*xPID - d*x(1);
23
24     % Simulate failure
25     torque = torqueD;
26     if ( k > kFail )
27         if ( sign(torque) == failSign )
28             torque = 0;
29         end
30     end
31     xP(:,k) = [x;torque + dist;torqueD];
32     x = RK4(@RHS,x,dT,0,inr,torque+dist);
33 end
34
35 t = (0:n-1)*dT;
36 yL = {'\theta (rad)', '\omega (rad/s)' 'Torque (Nm)'
37     '\Demand (Nm)'};
38 Plot2D(t,xP,'Time(s)',yL,'Failed_Thruster');
39
40 %% Right hand side
41 function xDot = RHS(x,~,inr,torque)
42 xDot = [x(2);torque/inr];
43
44 end

```



Example 16.6: Failed thruster. When the torque demand is the same as the failure sign, no control torque is produced.

16.4.4 Interface verification

Interface verification is used to make sure that the flight computer can communicate with the sensors and actuators on the spacecraft. Fig. 16.6 shows a general interface.

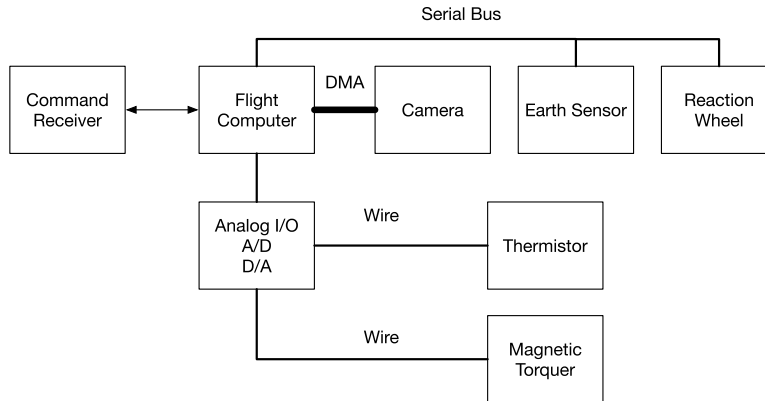


Figure 16.6 Hardware interfaces showing interface types. Direct memory access (DMA) is typically used for cameras although modern (terrestrial) serial interfaces are very fast.

Interfaces are generally through a serial data bus, like RS-422, SpaceWire, or MIL-STD 1553, or through single wires. Single wires provide analog signals. They may go through an analog-to-digital converter when coming from the sensor or a digital-to-analog converter when going to an actuator. For on/off signals a voltage pulse may be sufficient. Often the analog I/O is on a separate board. Modern microcontrollers may have multiple I/O pins on the motherboard. DMA is direct memory access and is useful if a sensor is transferring a large amount of data, like an image, to the flight computer.

It is rarely necessary to run a dynamical simulation to perform interface verification. If your flight software allows for commands, this is a clean way to exercise the interfaces.

16.4.5 Operator training

Operator training requires an interface much like the one shown in Fig. 16.5. Since the operators look at more than just ACS sensors, it is necessary to simulate the power, payload, communications, and thermal subsystem. Operators work from procedures. Once the procedures have been established, operators will work through them using the training simulation. It is not always necessary to have a dynamic simulation. In many cases, it is sufficient for the telemetry to just indicate that a command has been sent.

Fancier simulations are like computer games in which anomalies can be injected by a trainer. This was standard during the Apollo and Shuttle eras and presumably is used in the modern SpaceX and Boeing human-carrying spacecraft. For spacecraft with crews, the crew member is the operator. These may also involve operators so they can get quite complex. If a dynamical simulation is used, it is good to use the same one that was developed for the ACS. New simulation models may be necessary for the thermal and power subsystems as those subsystems do not always concern themselves with thermal- or power-system dynamics.

16.4.6 Anomaly investigations

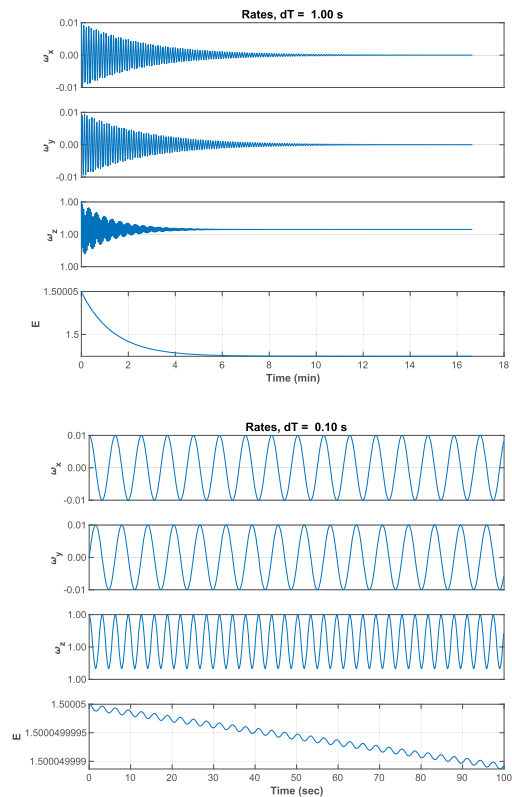
An anomaly may involve an ACS not performing as expected or a complete disaster like Mars Observer vanishing during its orbit insertion burn. In any situation, simulations can be part of the anomaly investigation. If an end-to-end simulation has been built then it can be used. In many cases, the smaller specialized simulations used for ACS development may be more useful. For example, if the star camera seems to be misbehaving the attitude determination simulations may be valuable. When doing an anomaly investigation it is important to be certain that your models can represent the issues that seem to be causing the problem. For example, the Hubble mirror anomaly clearly could not be studied by a pinhole-camera model of the optical system.

Before using the simulation tools it is important to be sure they model the spacecraft as observed on orbit. This can be difficult because telemetry data is limited. The sampling rate is usually low and the resolution of the data is also low. You may not be able to observe high-frequency effects since the sampling rates and resolution of the data are

```

1 %% Nutation
2
3 timeStep = [1 0.1];
4 n        = 1000;
5 inr      = diag([1;2;3]);
6
7 for j = 1:2
8
9     xP = zeros(8,n);
10
11     dT = timeStep(j);
12     x = [1;0;0;0;0.01;0;1];
13     for k = 1:n
14         omega = x(5:7);
15         e      = 0.5*omega*(inr*omega);
16         xP(:,k) = [x;e];
17         x       = RK4(@RHS,x,dT,0,inr);
18     end
19
20     t = (0:n-1)*dT;
21     yL = {'q_s' 'q_x' 'q_y' 'q_z' ...
22           '\omega_x' '\omega_y' '\omega_z' 'E' };
23
24     s = sprintf('Rates, dT=%4.2f s',dT);
25     TimeHistory(t,xP(5:8,:),yL(5:8),s);
26
27 end
28
29 %% Dynamics right-hand-side
30 function xDot = RHS(x,~,inr)
31 q      = x(1:4);
32 omega  = x(5:7);
33 omegaDot = -inr\cross(omega,inr*omega);
34 xDot    = [QIToBDot(q,omega);omegaDot];
35 end

```



Example 16.7: Artificial damping. The spacecraft appears to have damping with a time step of one second. The second simulation has the proper nutation response.

too low. Some spacecraft allow you to selectively download data at higher sample rates. This can be useful. If you have end-to-end simulations that produce simulated telemetry, these can help debug on-orbit issues.

16.5. Artificial damping

Simulations, because they are approximations, can give surprising results. Example 16.7 is a simulation of a spinning rigid body without any damping. The simulation is run at two time steps. At the larger time step, it appears that the angular rates damp. This is impossible and is entirely an artifact of the time step being too large.

Care needs to be taken with simulations to make certain that numerical artifacts are not misinterpreted as valid results.

References

- [1] D. Hough, IEEE 754-2019 - IEEE Standard for Floating-Point Arithmetic, Tech. Rep., IEEE Computer Society, July 2019.
- [2] M. Paluszek, The dynamics of the space shuttle orbiter with a flexible payload, *Acta Astronautica* 10 (4) (1983) 163–178.
- [3] M. Paluszek, Analysis of digital controllers using frequency-dependent describing functions, *IEEE Transactions on Automatic Control* 32 (7) (1987) 651–653.
- [4] Anonymous, Monte Carlo simulation, <https://www.ibm.com/cloud/learn/monte-carlo-simulation>, August 2020.